Automated RF Conformance Test

AccelerComm Graduate Recruitment Scenario for Test Report

Huy Viet Le

22/10/2026

Summary

This report presents the proposal of an automated RF conformance test in Python. A signal generator and spectrum analyser are configured over assumed SCPI/LAN comms. Power measurement is taken, compared against a two-sided spec window, and reported as human-readable and JSON output. The workflow is exercised by a unit-test suite against a fake link.

1. Overview

The objective is to design a Python solution that configures the instruments, executes a measurement, compares the result against a specification limit, calculates the compliance margin, reports in both human and machine-readable form, and demonstrates error handling and instrument cleanup. Low-level instrument communication functions already exist and the measured value is available from the analyser API the solution therefore does not attend to transport.

The system consists of a control computer as a remote SCPI client to two instruments: a signal generator that produces a controlled RF signal, and a spectrum analyser that measures it. The generator's output is cabled to the analyser's input; the control computer is not in the RF signal path, it only sends configuration commands and queries over LAN and reads back measured values.

2. Design And Implementation

2.1. System Setup and Equipment

The implementation uses three classes and one result dataclass, plus module-level functions. A fourth class attending specifically to the structure of the transport layer was considered but the idea was abandoned as such a class structure would hold no state, SCPI translation is one line of f-string formatting inside each domain method.

2.1.1. Instrument Class

Instrument — the base class. It owns the connection lifecycle (connect, close, __enter__, __exit__), delegates I/O (send, query) to the assumed link, and provides check_errors() to poll the instrument's SCPI error queue. It contains no transport code and no instrument-specific parameters.

2.1.2. SignalGenerator/SpectrumAnalyser Classes

Each adds only its own domain methods (configure, output_on/output_off, and measure_power) and its own SCPI placeholders. Connection, delegation, and cleanup are inherited.

2.1.3. Result Class

A dataclass holding the outcome of one measurement. It is the single source of truth for both report formats, so the human-readable and JSON outputs cannot disagree.

2.1.4. Module-level Functions

Consist of run_measurement (workflow), evaluate_compliance (pure comparison and margin), report_human and report_json (reporting).

2.2. Assumptions

Low-level communication functions (connect/send/query/close) already exist and that the measured value is available from the analyser API, so the solution does not reimplement transport. The Instrument class is a lifecycle wrapper around the assumed link: it pairs connect with close via __enter__/__exit__ so cleanup is guaranteed on exception, raises clear "not connected" and "already connected" errors on misuse, and keeps the single call to the assumed factory (open_link) in one place, so if the comms library changes only that method and the module-level declaration need updating. Instrument addressing follows a single VISA-style resource string, with the port encoded in the address and the connection timeout supplied as a default parameter on open_link. The comms layer is therefore assumed to provide open_link(address) returning a link with send, query, and close; SCPI commands are realistic placeholders whose exact mnemonics vary by instrument family; and :MEAS:POW? stands in for the measured value available from the analyser API, such that a real implementation would replace only measure_power(). Synchronisation assumes the analyser supports *OPC?, which is standard on spectrum analysers, and the plausibility bounds of −50 to +20 dBm used to reject implausible measurements are illustrative rather than derived from a specification. A single measurement per run is assumed; extending to a frequency sweep would loop the measure-and-compare step and produce a list of Result objects .

2.3. SCPI Translation

SCPI commands are ASCII strings, translation is formatting a Python value into the string and parsing to reply . The generator and analyser genuinely use different commands for the same physical quantity: the generator's frequency is :SOUR:FREQ, the analyser's centre frequency is :SENS:FREQ:CENT. A shared translator would therefore need a per-instrument lookup which is exactly what the subclass methods already are. It would add indirection without adding behaviour, and would hold no state. A translator class was considered but would have nothing to be responsible for, thus the idea was abandoned. The coupling that does matter, both instruments must use the same frequency, is made explicit by driving both configure calls from a single TestConfig.frequency_hz.

2.4. Workflow

The sequence in run_measurement is:

```
gen.configure(cfg.frequency_hz, cfg.setpoint_dbm)
sa.configure(cfg.frequency_hz, cfg.span_hz, cfg.rbw_hz)
gen.output_on()
sa.query("*OPC?")
return sa.measure_power()
```

Output-on is last so RF is only emitted once both instruments are configured. *OPC? blocks until the analyser has finished applying settings — exact rather than a guessed sleep.

3. Results and Reporting

The workflow returns a measured power in dBm, which is evaluated against a two-sided spec window (±1 dB around the −10 dBm setpoint in the default configuration). Margin is defined as the distance to the nearest limit: positive means inside the window with that much headroom, negative means outside by that amount. The verdict is PASS when margin ≥ 0, FAIL otherwise.

Results are reported in two forms, a human-readable form printed to stdout at the end of the run and a machine-readable form written to result.json, containing test name, timestamp, frequency, setpoint, measured value, both limits, margin, and verdict.

The process exit code reflects the outcome (0 = PASS, 1 = FAIL, 2 = error), so the script can be used in a CI pipeline without parsing output.

4. Error Handling And Cleanup

Cleanup is guaranteed by the context-manager protocol. SignalGenerator.__exit__ additionally disables RF output before closing, so the generator is never left radiating. send/query raise clear errors if called before connect; close is idempotent.

After every configuration step, the instrument's SCPI error queue is polled via SYST:ERR?. An instrument can silently ignore a malformed SCPI command rather than raising on the comms layer, so this check catches a class of failure that would otherwise go unnoticed. Measured values are range-checked before being turned into a verdict, so an implausible reading cannot become a PASS.

Retries were left out deliberately. A conformance test should report the measurement it got, not mask an intermittent fault. Similarly, a session that drops mid-test aborts the run: the intent is to fail loudly and visibly rather than to hide flakiness.

5. Testing

The workflow is exercised against a fake instrument link in test_rf_conformance_test.py, with no real hardware required. The suite covers: the happy path; cleanup running on an exception; guards against send/query before connection and double-connection; idempotent close; per-instrument routing of SCPI commands; the full measurement flow; the four compliance edge cases (PASS, fail below, fail above, on-limit); plausibility rejection of an implausible measurement; and JSON validity.

6. Limitations and Possible Improvements

*OPC? synchronisation waits for settings to be applied, but does not guarantee that the read value comes from a new acquisition; a production version would send INIT:IMM and wait on *OPC? before reading the marker.

AI-Assisted Development

AI was used to explain and bridge the knowledge gaps in SCPI language, or clarify the priorities of RF Conformance testing in industry to provide a high-level idea of the challenge. AI also helped identifying some of the needed tests for some of the edge cases or consult appropriate assumptions to be made relevant to industry standards and practices. The design decisions, structure of the code, and their final form are my own.

References

1. SCPI Consortium, Standard Commands for Programmable Instruments (SCPI), Vol. 1: Syntax and Style, 1999.Agrawal, G. P. (2021). Fiber-Optic Communication Systems. Germany: Wiley.
2. GPP, TS 38.521-1: 5G; NR; User Equipment (UE) conformance specification; Radio transmission and reception; Part 1: Range 1 standalone, version 18.5.0, Release 18, ETSI TS 138 521-1, February 2025. Available at: https://www.etsi.org/deliver/etsi_ts/138500_138599/13852101/18.05.00_60/ts_13852101v180500p.pdf


rf_conformance_test.py

APPENDIX 1 - RF CONFORMANCE TEST MODULE - rf_conformance_test.py

```
"""
rf_conformance_test.py

Automated RF conformance test for a signal generator + spectrum analyser pair.
The script runs on a control computer as a remote SCPI client over LAN. The
low-level comms layer (connect/send/query/close) is assumed to already exist
Workflow: configure both instruments, enable RF, synchronise, measure,
compare against a two-sided spec window, report PASS/FAIL with margin in
human-readable and JSON forms. See the accompanying design note for the
rationale behind each design choice.
"""
import json
import os
import sys
from dataclasses import dataclass, asdict
from datetime import datetime, timezone

@dataclass(frozen=True)
class TestConfig:
    test_name: str = "rf_power_conformance"
    frequency_hz: float = 2.4e9
    setpoint_dbm: float = -10.0
    span_hz: float = 10e6
    rbw_hz: float = 100e3
    lower_limit_dbm: float = -11.0
    upper_limit_dbm: float = -9.0

def open_link(address: str, timeout_s: float = 5.0):
    """Open a SCPI session to `address` and return a link object.
    Assumed to be provided by the existing instrument comms layer.
    - `address`: VISA-style resource string identifying one instrument.
    - `timeout_s`: connection timeout, delegated to the underlying layer.
    The returned link is expected to provide .send(cmd), .query(cmd) -> str,
    and .close().
    """
    raise NotImplementedError("Provided by the existing instrument comms layer")
# ---------------------------------------------------------------------------
# Instrument: lifecycle wrapper around one assumed comms link.
# ---------------------------------------------------------------------------
class Instrument:
    def __init__(self, address: str, name: str) -> None:
        self.address = address
        self.name = name
        self._link = None
    # -- lifecycle ------------------------------------------------------
    def connect(self) -> None:
        """Open the SCPI session.
        Raises if already connected. If open_link() fails, its exception
```

```
        propagates and self._link stays None, leaving the object disconnected.
        """
        if self._link is not None:
            raise RuntimeError(f"{self.name}: already connected")
        self._link = open_link(self.address)   # module-level assumed factory

    def close(self) -> None:
        """Close the SCPI session if open. Safe to call more than once."""
        if self._link is not None:
            self._link.close()
            self._link = None

    def __enter__(self) -> "Instrument":
        self.connect()
        return self

    def __exit__(self, exc_type, exc, tb) -> None:
        # Runs whether the body succeeded, failed, or raised.
        self.close()

    # -- delegation to the assumed link ---------------------------------
    def send(self, cmd: str) -> None:
        """Send a write-only SCPI command."""
        if self._link is None:
            raise RuntimeError(f"{self.name}: not connected")
        self._link.send(cmd)

    def query(self, cmd: str) -> str:
        """Send a SCPI query and return the raw response string."""
        if self._link is None:
            raise RuntimeError(f"{self.name}: not connected")
        return self._link.query(cmd)

    def check_errors(self) -> None:
        """Raise if the instrument's error queue is non-empty."""
        response = self.query("SYST:ERR?").strip()
        code = response.split(",", 1)[0].strip()
        if code not in ("0", "+0"):
            raise RuntimeError(f"{self.name}: SCPI error: {response}")

# ---------------------------------------------------------------------------
# Instrument-specific subclasses.
#
# Each adds ONLY its own domain methods — the SCPI placeholder strings.
# Connection, send/query, and cleanup are inherited from Instrument.
# SCPI command text is a realistic placeholder (exact mnemonics vary by model).
# ---------------------------------------------------------------------------
class SignalGenerator(Instrument):
    """Stimulus source: produces a controlled RF signal."""
    def configure(self, freq_hz: float, level_dbm: float) -> None:
        """Reset to a known state and apply the stimulus settings."""
        self.send("*RST")                      # known starting state
```

```
        self.send(f":SOUR:FREQ {freq_hz}")     # placeholder
        self.send(f":SOUR:POW {level_dbm}")    # placeholder
        self.check_errors()
        # RF output is enabled separately so cleanup can pair it.

    def output_on(self) -> None:
        """Enable the RF output."""
        self.send(":OUTP ON")                  # placeholder
        self.check_errors()

    def output_off(self) -> None:
        """Disable the RF output."""
        self.send(":OUTP OFF")                 # placeholder

    def __exit__(self, exc_type, exc, tb):
       try:
           if self._link is not None:
               self.output_off()
       except Exception:
           print(f"[WARN] {self.name}: could not confirm RF output off")
       finally:
           super().__exit__(exc_type, exc, tb)

class SpectrumAnalyser(Instrument):
    """Measurement device: observes the RF signal."""
    def configure(self, centre_hz: float, span_hz: float, rbw_hz: float) -> None:
        """Reset to a known state and apply the measurement-window settings."""
        self.send("*RST")                          # known starting state
        self.send(f":SENS:FREQ:CENT {centre_hz}")  # placeholder
        self.send(f":SENS:FREQ:SPAN {span_hz}")    # placeholder
        self.send(f":SENS:BAND:RES {rbw_hz}")      # placeholder

    def measure_power(self) -> float:
        """Return the measured power in dBm."""
        value = float(self.query(":MEAS:POW?"))
        if not (-50.0 <= value <= 20.0):
            raise ValueError(f"implausible measurement: {value} dBm")
        return value

def run_measurement(gen: SignalGenerator, sa: SpectrumAnalyser, cfg: TestConfig) -> float:
    """Configure both instruments, enable RF, synchronise, and measure.
    Returns the measured power in dBm.
    Assumes both instruments are already connected (inside their `with` blocks).
    """
    gen.configure(cfg.frequency_hz, cfg.setpoint_dbm)
    sa.configure(cfg.frequency_hz, cfg.span_hz, cfg.rbw_hz)
    gen.output_on()
    sa.query("*OPC?")          # block until settings applied / sweep done
    return sa.measure_power()
# ---------------------------------------------------------------------------
# Compliance evaluation and reporting.
#
```

```python
# Pure functions, deliberately outside the instrument classes: none of these
# is an instrument concern. evaluate_compliance() has no side effects, so it
# can be tested independently of any hardware.
# ---------------------------------------------------------------------------
@dataclass(frozen=True)
class Result:
    """Outcome of one conformance measurement."""
    test_name: str
    timestamp: str
    frequency_hz: float
    setpoint_dbm: float
    measured_dbm: float
    lower_limit_dbm: float
    upper_limit_dbm: float
    margin_db: float
    verdict: str

def evaluate_compliance(
    measured_dbm: float,
    lower_dbm: float,
    upper_dbm: float,
    freq_hz: float,
    setpoint_dbm: float,
    test_name: str = "rf_power_conformance",
) -> Result:
    """Compare a measured value against a two-sided spec window.
    Margin convention: distance from the measured value to the NEAREST limit.
    Positive margin  -> inside the window (PASS), with that much headroom.
    Negative margin  -> outside the window (FAIL), by that amount.
    """
    margin = min(measured_dbm - lower_dbm, upper_dbm - measured_dbm)
    verdict = "PASS" if margin >= 0 else "FAIL"
    return Result(
        test_name=test_name,
        timestamp=datetime.now(timezone.utc).isoformat(timespec="seconds"),
        frequency_hz=freq_hz,
        setpoint_dbm=setpoint_dbm,
        measured_dbm=measured_dbm,
        lower_limit_dbm=lower_dbm,
        upper_limit_dbm=upper_dbm,
        margin_db=round(margin, 3),
        verdict=verdict,
    )

def report_human(result: Result) -> None:
    """Print a human-readable summary of the result."""
    print()
    print(f"  Test      : {result.test_name}")
    print(f"  Frequency : {result.frequency_hz / 1e9:.3f} GHz")
    print(f"  Setpoint  : {result.setpoint_dbm:+.1f} dBm")
    print(f"  Measured  : {result.measured_dbm:+.2f} dBm")
    print(f"  Limits    : {result.lower_limit_dbm:+.1f} .. {result.upper_limit_dbm:+.1f} dBm")
```

```
    print(f"  Margin    : {result.margin_db:+.2f} dB")
    print(f"  Verdict   : {result.verdict}")
    print()

def report_json(result: Result, path: str) -> None:
    """Write the result as JSON for machine consumption."""
    with open(path, "w") as f:
        json.dump(asdict(result), f, indent=2)
    print(f"[report] machine-readable result written to {path}")

def main(gen_address: str, sa_address: str) -> int:
    """Run the conformance test end-to-end. Returns a process exit code."""
    cfg = TestConfig()          # defaults live in the dataclass
    try:
        with SignalGenerator(gen_address, "generator") as gen, \
             SpectrumAnalyser(sa_address, "analyser") as sa:
            measured = run_measurement(gen, sa, cfg)
            result = evaluate_compliance(
                measured, cfg.lower_limit_dbm, cfg.upper_limit_dbm,
                cfg.frequency_hz, cfg.setpoint_dbm, cfg.test_name,
            )
            report_human(result)
            report_json(result, "result.json")
            return 0 if result.verdict == "PASS" else 1
    except Exception as exc:
        print(f"[ERROR] {type(exc).__name__}: {exc}")
        return 2

if __name__ == "__main__":
    gen_address = os.environ.get("RF_GEN_ADDRESS", "")
    sa_address = os.environ.get("RF_SA_ADDRESS", "")

    if not gen_address or not sa_address:
        print("ERROR: set RF_GEN_ADDRESS and RF_SA_ADDRESS")
        sys.exit(2)
    sys.exit(main(gen_address, sa_address))
```

APPENDIX 2 - TEST FILE - testing.py

testing.py

```
"""
test_instrument.py

Minimal demonstration/test of the Instrument base class.
Uses a FAKE comms link so the class can be exercised without hardware.
.
"""
import instrument as inst_mod
from instrument import (Instrument, SignalGenerator, SpectrumAnalyser,
    run_measurement, evaluate_compliance, TestConfig, Result,
)
# ---------------------------------------------------------------------------
# Test double for the assumed comms link.
# ---------------------------------------------------------------------------

class _FakeLink:
    """Stand-in for the object returned by the assumed open_link()."""
    def __init__(self, address):
        self.address = address
        self.open = True
        self.sent = []          # record commands, so tests can assert on them
        self.queried = []

    def send(self, cmd):
        if not self.open:
            raise RuntimeError("send on closed fake link")
        self.sent.append(cmd)
        print(f"[fake] send  {self.address!r} <- {cmd!r}")

    def query(self, cmd):
        if not self.open:
            raise RuntimeError("query on closed fake link")
        self.queried.append(cmd)
        print(f"[fake] query {self.address!r} <- {cmd!r}")
        if cmd == ":MEAS:POW?":
            return "-10.3"
        if cmd == "SYST:ERR?":
            return '+0,"No error"'
        if cmd == "*OPC?":
            return "1"
        return "FAKE-IDN"

    def close(self):
        print(f"[fake] close {self.address!r}")
        self.open = False

def _fake_open_link(address, timeout_s=5.0):
    print(f"[fake] open_link({address!r}, timeout_s={timeout_s})")
```

```
    return _FakeLink(address)


def _install_fake_comms():
    """Replace the assumed comms layer with the fake for the duration of a test."""
    inst_mod.open_link = _fake_open_link
# ---------------------------------------------------------------------------
# Tests
# ---------------------------------------------------------------------------

def test_happy_path():
    """Open, send, query, close — the normal flow."""
    _install_fake_comms()
    with Instrument("FAKE::generator", "generator") as inst:
        inst.send(":SOUR:FREQ 2.4e9")
        idn = inst.query("*IDN?")
        assert idn == "FAKE-IDN"
        assert inst._link is not None           # still connected inside `with`
    assert inst._link is None                   # closed by __exit__
    print("[test] happy path: OK")

def test_cleanup_runs_on_exception():
    """Close must run even when the with-body raises."""
    _install_fake_comms()
    inst = Instrument("FAKE::analyser", "analyser")
    try:
        with inst:
            raise RuntimeError("simulated failure")
    except RuntimeError:
        pass
    assert inst._link is None                   # __exit__ closed it
    print("[test] cleanup on exception: OK")

def test_send_before_connect_fails_clearly():
    """send() on a disconnected instrument raises a clear error."""
    _install_fake_comms()
    inst = Instrument("FAKE::generator", "generator")
    try:
        inst.send(":SOUR:FREQ 2.4e9")
        assert False, "expected RuntimeError"
    except RuntimeError as e:
        assert "not connected" in str(e)
    print("[test] send before connect: OK")

def test_query_before_connect_fails_clearly():
    """query() on a disconnected instrument raises a clear error."""
    _install_fake_comms()
    inst = Instrument("FAKE::generator", "generator")
    try:
        inst.query("*IDN?")
        assert False, "expected RuntimeError"
    except RuntimeError as e:
```

```
        assert "not connected" in str(e)
    print("[test] query before connect: OK")
```

```
def test_double_connect_rejected():
    """Calling connect() twice must raise, not leak a second session."""
    _install_fake_comms()
    inst = Instrument("FAKE::generator", "generator")
    inst.connect()
    try:
        inst.connect()
        assert False, "expected RuntimeError"
    except RuntimeError as e:
        assert "already connected" in str(e)
    inst.close()
    print("[test] double connect rejected: OK")

def test_close_is_idempotent():
    """close() before connect, then twice after connect, must all be safe."""
    _install_fake_comms()
    inst = Instrument("FAKE::generator", "generator")
    inst.close()          # never connected — must be a no-op
    inst.connect()
    inst.close()
    inst.close()          # already closed — must be a no-op
    print("[test] close is idempotent: OK")

def test_commands_reach_the_link_in_order():
    """What the Instrument sends is exactly what reaches the assumed link."""
    _install_fake_comms()
    with Instrument("FAKE::generator", "generator") as inst:
        inst.send(":SOUR:FREQ 2.4e9")
        inst.send(":SOUR:POW -10")
        assert inst._link.sent == [":SOUR:FREQ 2.4e9", ":SOUR:POW -10"]
    print("[test] commands reach link in order: OK")

def test_generator_commands_reach_generator_link_only():
    """Generator config sends exactly the expected SCPI, to its own link."""
    _install_fake_comms()
    with SignalGenerator("FAKE::gen", "generator") as gen:
        gen.configure(2.4e9, -10.0)
        gen.output_on()
        assert gen._link.sent == [
            "*RST",
            ":SOUR:FREQ 2400000000.0",
            ":SOUR:POW -10.0",
            ":OUTP ON",
        ]
    print("[test] generator commands reach generator link only: OK")

def test_analyser_commands_reach_analyser_link_only():
    """Analyser config sends exactly the expected SCPI, to its own link."""
```

```
    _install_fake_comms()
    with SpectrumAnalyser("FAKE::sa", "analyser") as sa:
        sa.configure(2.4e9, 10e6, 100e3)
        assert sa._link.sent == [
            "*RST",
            ":SENS:FREQ:CENT 2400000000.0",
            ":SENS:FREQ:SPAN 10000000.0",
            ":SENS:BAND:RES 100000.0",
        ]
    print("[test] analyser commands reach analyser link only: OK")

def test_measurement_flow():
    """Full flow: configure → output on → synchronise → measure; returns a number."""
    _install_fake_comms()
    with SignalGenerator("FAKE::gen", "generator") as gen, \
         SpectrumAnalyser("FAKE::sa", "analyser") as sa:
        measured = run_measurement(gen, sa, TestConfig())
        assert measured == -10.3
        # spot-check commands reached the right instruments
        assert gen._link.sent == [
            "*RST",
            ":SOUR:FREQ 2400000000.0",
            ":SOUR:POW -10.0",
            ":OUTP ON",
        ]
        assert sa._link.sent == [
            "*RST",
            ":SENS:FREQ:CENT 2400000000.0",
            ":SENS:FREQ:SPAN 10000000.0",
            ":SENS:BAND:RES 100000.0",
        ]
        # queried is now interleaved with SYST:ERR? checks — assert membership
        assert ":MEAS:POW?" in sa._link.queried
    print(f"[test] measurement flow: OK (measured={measured} dBm)")

def test_compliance_pass():
    """A measured value comfortably inside the window -> PASS."""
    r = evaluate_compliance(-10.3, -11.0, -9.0, 2.4e9, -10.0)
    assert r.verdict == "PASS"
    assert r.margin_db == 0.7
    print("[test] compliance PASS: OK")

def test_compliance_fail_below():
    """A measured value below the lower limit -> FAIL, negative margin."""
    r = evaluate_compliance(-11.5, -11.0, -9.0, 2.4e9, -10.0)
    assert r.verdict == "FAIL"
    assert r.margin_db == -0.5
    print("[test] compliance FAIL (below): OK")

def test_compliance_fail_above():
    """A measured value above the upper limit -> FAIL, negative margin."""
    r = evaluate_compliance(-8.5, -11.0, -9.0, 2.4e9, -10.0)
```

```
    assert r.verdict == "FAIL"
    assert r.margin_db == -0.5
    print("[test] compliance FAIL (above): OK")
```

```
def test_compliance_on_limit_passes():
    """Exactly on a limit counts as PASS (margin 0)."""
    r = evaluate_compliance(-11.0, -11.0, -9.0, 2.4e9, -10.0)
    assert r.verdict == "PASS"
    assert r.margin_db == 0.0
    print("[test] compliance on limit: OK")

def test_report_json_writes_valid_file():
    """JSON reporter writes a valid, parseable file."""
    import json, os, tempfile
    r = evaluate_compliance(-10.3, -11.0, -9.0, 2.4e9, -10.0)
    with tempfile.TemporaryDirectory() as d:
        path = os.path.join(d, "result.json")
        from instrument import report_json
        report_json(r, path)
        with open(path) as f:
            data = json.load(f)
        assert data["verdict"] == "PASS"
        assert data["margin_db"] == 0.7
        assert data["measured_dbm"] == -10.3
    print("[test] JSON report: OK")

def test_generator_rf_off_on_exit():
    """RF output is disabled when the generator's with-block exits."""
    _install_fake_comms()
    gen = SignalGenerator("FAKE::gen", "generator")
    with gen:
        gen.output_on()
        link = gen._link            # keep a reference before close resets it
    assert link.sent[-1] == ":OUTP OFF"
    print("[test] generator RF off on exit: OK")

def test_measurement_implausible_raises():
    """A physically implausible measurement raises rather than producing a verdict."""
    _install_fake_comms()
    class _ImplausibleLink(_FakeLink):
        def query(self, cmd):
            if cmd == ":MEAS:POW?":
                return "999.0"
            return super().query(cmd)

    inst_mod.open_link = lambda address, timeout_s=5.0: _ImplausibleLink(address)
    with SpectrumAnalyser("FAKE::sa", "analyser") as sa:
        try:
            sa.measure_power()
            assert False, "expected ValueError"
        except ValueError as e:
```

```
            assert "implausible" in str(e)
    print("[test] implausible measurement raises: OK")

def test_scpi_error_queue_checked():
    """A non-zero SYST:ERR? code raises an error."""
    _install_fake_comms()

    class _ErrorLink(_FakeLink):
        def query(self, cmd):
            if cmd == "SYST:ERR?":
                return '-222,"Data out of range"'
            return super().query(cmd)

    inst_mod.open_link = lambda address, timeout_s=5.0: _ErrorLink(address)

    with Instrument("FAKE::gen", "generator") as inst:
        try:
            inst.check_errors()
            assert False, "expected RuntimeError"
        except RuntimeError as e:
            assert "SCPI error" in str(e)
    print("[test] SCPI error queue checked: OK")
# ---------------------------------------------------------------------------

if __name__ == "__main__":
    tests = [
        test_happy_path,
        test_cleanup_runs_on_exception,
        test_send_before_connect_fails_clearly,
        test_query_before_connect_fails_clearly,
        test_double_connect_rejected,
        test_close_is_idempotent,
        test_commands_reach_the_link_in_order,
        test_generator_commands_reach_generator_link_only,
        test_analyser_commands_reach_analyser_link_only,
        test_measurement_flow,
        test_compliance_pass,
        test_compliance_fail_below,
        test_compliance_fail_above,
        test_compliance_on_limit_passes,
        test_report_json_writes_valid_file,
        test_generator_rf_off_on_exit,
        test_measurement_implausible_raises,
        test_scpi_error_queue_checked,
    ]
    for t in tests:
        t()
    print(f"\n[test] all {len(tests)} checks passed")
```